McMaster University
Faculty of Engineering
COMPSCI 4ZP6 - Capstone Project

Final Reflection Document

ARDEN

AI-Driven Game Narrative Editor

Team 30

Cathy Xu	xu518@mcmaster.ca
Duzhi Wu	wud72@mcmaster.ca
Moein Roghani	roghanim@mcmaster.ca
Xingjian Zheng	zhengx68@mcmaster.ca

Website: arden-dev.com

GitHub: github.com/ARDEN-MC

arden-studio-web · arden-studio-api · arden-studio-data-service · arden-unity

Version 0
April 4, 2026

Capstone Instructor: Dr. Mehdi Moradi

Table of Contents

1 Why Is ARDEN Highly Complex?

ARDEN Studio extends Microsoft's GENEVA research (IEEE CoG 2024) from a proof-of-concept LLM narrative generator into a full production platform. Building it required solving challenges across four distinct engineering domains simultaneously.

Multi-Agent AI Orchestration. We built a fully custom agent architecture using only LangGraph primitives—no external agent frameworks or prebuilt multi-agent libraries. The system is built on two distinct agent bases, each with its own architecture (Appendix B, Appendix C):

Supervisor Agent Base uses LangGraph's `create_react_agent` (ReAct pattern) with a `MemorySaver` checkpointer, giving it persistent conversation memory across the entire session. Its tools are not file tools—they are delegation functions (`delegate_to_lore_bible_expert`, `delegate_to_narrative_graph_expert`) that invoke sub-agents, composing self-contained instructions from the conversation context.

Sub-Agent Base (`precontex_agent_base`) is a custom `StateGraph` we built from scratch. Its graph flow is: `context_node` → `agent` → `tools` → `agent` (loops until no tool calls) → `END`. The `context_node` runs a pre-context function that fetches the current project data (lore or storyline JSON) from the database and injects it as a `SystemMessage` before the LLM ever sees the request. The base also validates chat history integrity by ensuring every tool call has a matching `ToolMessage`, preventing malformed conversation state from reaching the LLM.

These two bases compose into an agentic workflow (Appendix D) where the supervisor delegates to sub-agents that are SMEs (Subject Matter Experts) of their own domain—Lore Bible or Narrative Graph. This design deliberately avoids the traditional single-agent-with-all-tools pattern, where one LLM bears the full cognitive load. Instead, ARDEN distributes work across **three separate LLM instances**, each with its own prompt engineering and model configuration: the supervisor (creative direction, temperature 1.0) and two experts (precise data operations, temperature 0.85). Each layer—prompt, model, and tools—is independently configured per agent.

Tool-Augmented Generation with Verification. Instead of having the LLM produce raw text, our agents operate on persistent structured data through three custom tools: `ReadFileTool` (returns file content with line numbers for precise reference), `EditFileTool` (atomic sequential find-and-replace with two-phase simulation and rollback), and `WriteFileTool` (full JSON overwrite with pre-write validation). The key innovation is `EditFileTool`: it first simulates all edits against progressively-modified content, then applies them only if the full sequence is valid, with automatic rollback on failure. Every write operation is verified via read-back comparison against the saved content. This eliminated an entire class of data corruption bugs that plague direct LLM generation approaches.

Three-Tier Distributed System. We engineered and deployed four containerized services—React SPA, FastAPI Python backend, Node.js/Express data service, and PostgreSQL—orchestrated with Docker Compose on a Linux VM hosted on DigitalOcean. Coordinating authentication, session management, AI pipeline state, and data persistence across three

application layers written in different languages was a significant systems challenge.

Custom Graph Layout Algorithm. The narrative graph editor uses a hand-rolled layered DAG layout algorithm—no external graph libraries (dagre, ELK, React Flow) are involved. It implements a modified Kahn's topological sort with parent-readiness gating and vertical centering per layer. Full pseudocode is in Appendix A.

Real-Time AI-to-UI Integration. Both the AI and the user can modify the same narrative graph, so we solved every concurrency edge case: *User edits during AI work?* UI controls are disabled while the AI pipeline is active. *Overlapping changes?* Pending user edits are flushed to the database before AI begins. *Stale UI after AI writes?* The frontend cancels debounced saves and performs a full project reload when the AI finishes.

Unity Narrative Engine Plugin. We built a Unity plugin (`NarrativeTool`) that consumes ARDEN's exported DAG and lore JSON and provides a playable narrative runtime. It is structured across four assemblies (Core and Runtime are pure C#, Unity and Unity.Editor add engine integration). Key components include a DAG traversal state machine, a variable store with subscriptions and save/load snapshots, an expression evaluator for edge conditions, a lore bible loader with strongly-typed entity models, and editor tools (JSON Simulator, Graph Visualizer with live node tracking). The plugin ships with NUnit tests.

2 Achievements

AI Topline Metric. Every AI write operation passes two-phase edit simulation, JSON validation, and read-back verification before being reported as successful—these safeguards are architectural and run on every tool invocation. The entire agent pipeline is instrumented with Langfuse for observability, allowing us to monitor workflow correctness, latency, and accuracy in production. Throughout development and production use, we observed zero data corruption incidents from AI-generated writes.

What This App Does Uniquely. ARDEN is the only tool we are aware of that unifies AI-assisted narrative generation with a live visual graph editor in a single interface. Existing narrative tools (Twine, Ink, articy:draft) require entirely manual graph construction. Conversational AI tools (ChatGPT, Claude) produce ephemeral text with no persistent project state. ARDEN bridges both: users converse naturally with an AI assistant that directly reads, creates, and modifies narrative graphs and world-building data, with every change instantly reflected in the visual editor. Beyond the web platform, ARDEN delivers a complete Unity plugin (`NarrativeTool`) that takes the exported DAG and lore JSON and provides a runtime narrative engine—with a state machine, variable system, condition evaluation, story tracking, and editor tools—so AI-generated narratives are directly playable in Unity without any manual conversion. This end-to-end pipeline, from AI-assisted authoring to game engine playback, takes Microsoft's GENEVA research concept and turns it into a usable, deployed product with sessions, authentication, and structured data export.

3 P0 and P1 Requirements Completion

3.1 Priority 0 — Minimum Viable Product

Requirement	%	Requirement	%
React multi-panel dashboard with AI Assistant and Graph Editor	100	Node-based story creation (start, dialogue, choice, end) with drag-and-drop	100
Python backend with LangGraph for AI generation and conversation context	100	Node.js database service for PostgreSQL integration	100
PostgreSQL for users, projects, node data, lore, and relationships	100	Session-based state preservation across multiple sessions	100
JSON export of narrative graphs for Unity integration	100	Secure authentication and project-based access control	100
Developer documentation covering API routes, DB schema, integration	75		

P0 Overall: 97%. All MVP requirements implemented and deployed to production; one requirement delivered in a different format than originally specified.

Below 80% — Developer documentation (75%): We pivoted from Swagger/OpenAPI to documenting all API routes and DB schemas as tables in the Design document. The content is complete but not in the originally planned interactive format.

3.2 Priority 1 — Next Feature Set

Requirement	%	Requirement	%
Inspector Panel for text content, metadata, and node properties	100	Lore Bible interface for characters, locations, events, world-building	100
Real-time bidirectional sync between Graph Editor and backend	100	Validation and linting for narrative contradictions and plot holes	100
Version tracking and rollback capabilities	0	Incremental file modification through editing algorithms	100
Backend endpoints for project import/export and workspace management	100	Enhanced Data Service with advanced querying and connection pooling	100
Expanded backend testing coverage	100		

P1 Overall: 89% (8 of 9 requirements at 100%).

Below 80% — Version tracking and rollback (0%): Implementing this requires a multi-version state management layer synchronized with LangGraph conversation memory—a substantial extension beyond current scope. Deferred to future work.

Note on validation (100%, different approach): The SRS specified regex-based validation. We replaced this with AI-based validation: the tool framework enforces structural validity via JSON validation and read-back verification on every write. Since users never edit raw JSON, regex-based validation is unnecessary.

4 Highlights and Lowlights

Highlights

- The multi-agent AI system works reliably in production—users can have natural creative conversations and the supervisor correctly delegates to the right expert, producing structurally valid narrative content.
- Our agent architecture—supervisor owns memory and intent, stateless experts own tools and execution, `precontex_agent_base` bridges the gap via context injection—avoids shared memory complexity while maintaining full project awareness at every step.
- Full Docker Compose deployment on a DigitalOcean Linux VM with four containerized services running stably throughout the semester.

Lowlights

- Version tracking and rollback was the most significant feature we had to cut. The LangGraph memory system does not natively support branching or snapshotting project data, and building this from scratch was beyond our timeline.
- AI response latency occasionally reaches 120-300 seconds for complex multi-agent workflows with multiple tool calls, which can disrupt the conversational flow.

5 What We Learnt

Delegation design is everything in multi-agent systems. Stateless agents cannot infer context—every detail must be explicitly passed. We iterated heavily on prompt engineering to make the supervisor produce self-contained delegation instructions.

Tools beat raw generation. Structured tools with built-in validation produce far more reliable results than asking an LLM to generate complete JSON. The atomic edit-with-rollback pattern was our most impactful architectural decision.

Scope trade-offs are unavoidable. Replacing regex with AI-based validation, deferring version tracking, and prioritizing stability over feature count were deliberate, technically justified choices.

Deploying a distributed system teaches things local development cannot. Four services on a live DigitalOcean VM surfaced challenges we never hit locally: DNS configuration, container endpoint routing, hostname resolution, SSL setup, and cross-service environment variable management.

Appendix: System Design Artifacts

The following diagrams are from ARDEN's Design, Validation & Verification document and illustrate the system's architecture and key interaction flows.

A. Custom Layered DAG Layout Algorithm

The narrative graph editor in `GraphPanel.js` uses a fully custom layered DAG layout algorithm with no external graph layout libraries. The algorithm assigns nodes to horizontal depth layers via a modified topological sort, then vertically centers each layer for balanced visual output.

Algorithm: AutoLayoutNodes

Input: Set of nodes N, set of directed edges E
Output: Updated node positions (x, y) for all topologically connected nodes

```
function AutoLayoutNodes(N, E):
    // Phase 1 | Build topology
    for each node n in N:
        children[n] ←
        inDegree[n] ← 0
    for each edge (u, v) in E:
        children[u] ← children[u]  {v}
        inDegree[v] ← inDegree[v] + 1

    // Phase 2 | Layer assignment (modified Kahn's algorithm)
    layers ← []
    visited ←
    queue ← {n  N | inDegree[n] = 0}     // start with root nodes
    if queue =  then queue ← {N[0]}       // fallback: pick first node

    while queue  :
        layers.append(queue)
        visited ← visited  queue
        next ←
        for each node u in queue:
            for each child v in children[u]:
                if v  visited  v  next:
                    // Parent-readiness gate: only advance when
                    // ALL parents of v have been placed
                    if  edge (w, v)  E : w  visited then
                        next ← next  {v}
        queue ← next
```

```
// Phase 3 | Position assignment with vertical centering
for each layer L at column index col:
    totalHeight ← |L| × NODE_H + (|L| - 1) × Y_GAP
    startY ← OFFSET_Y + max(0, 300 - totalHeight / 2)
    for each node n at row index row in L:
        n.x ← OFFSET_X + col × X_GAP
        n.y ← startY + row × (NODE_H + Y_GAP)

// Nodes with no edges retain their original positions
return updated N
```

Key Design Decisions:

- **Parent-readiness gating:** A node is only placed in the next layer when *all* of its incoming edges originate from already-placed nodes. This prevents a node from appearing in a layer before one of its parents, preserving the DAG's visual dependency ordering.
- **Vertical centering:** Each layer is vertically centered around a midpoint, producing a balanced visual layout regardless of layer sizes. Layers with fewer nodes are pushed toward the vertical center rather than the top.
- **Disconnected node preservation:** Nodes with no edges (isolated or user-positioned) retain their original coordinates and are not affected by the layout algorithm.
- **Incremental re-layout:** When new nodes arrive from AI generation, the algorithm runs on the combined set of existing and new nodes but preserves local positions for previously-placed nodes, only computing positions for newly added ones.

Complexity: $O(|N| + |E|)$ for the topological sort phase, with an additional $O(|E|)$ factor per node for the parent-readiness check, yielding $O(|N| \cdot |E|)$ worst case. In practice, narrative graphs are sparse ($|E| \approx |N|$), so the algorithm runs in near-linear time.

B. `precontex_agent_base` Architecture

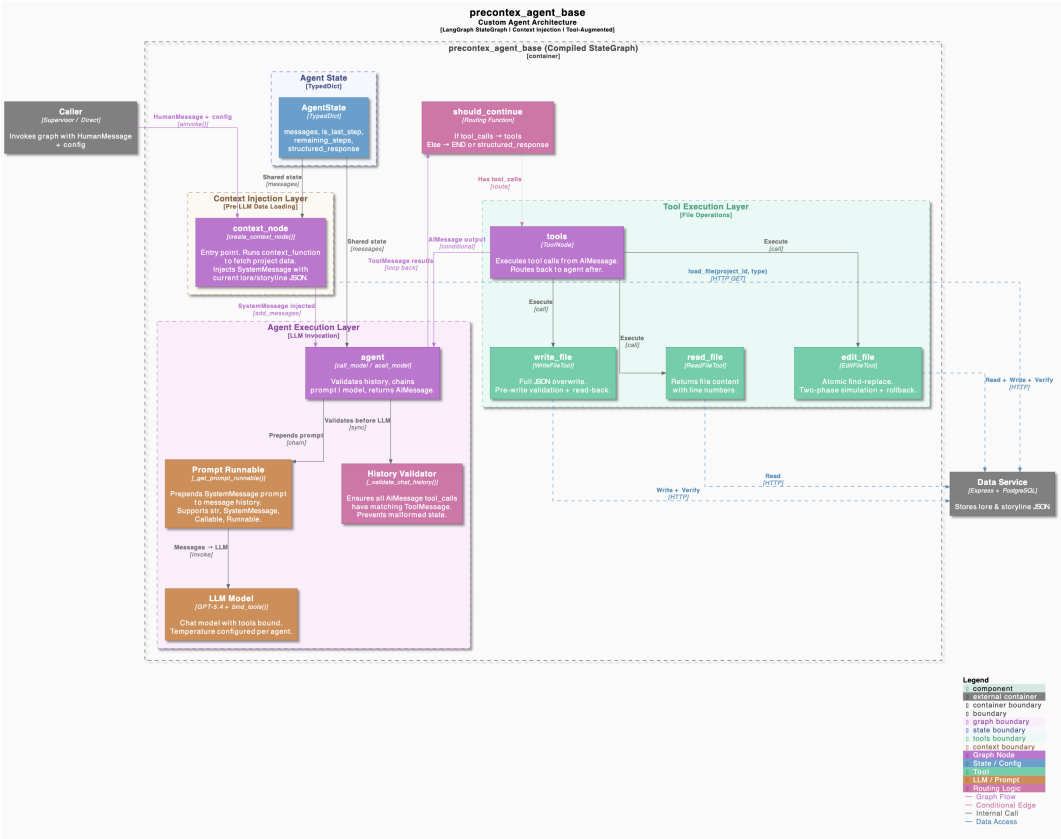

Figure 1: C4 Component Diagram: `precontex_agent_base` — a custom `StateGraph` built from scratch using LangGraph primitives. The graph flows through a context injection node (pre-loads project data as a `SystemMessage`), an agent node (prompt + LLM with chat history validation), and a tool execution node (`read_file`, `edit_file`, `write_file`) that loops back until no tool calls remain. Sub-agents are stateless; each receives a single instruction and executes in isolation.

C. Supervisor (`create_react_agent`) Architecture

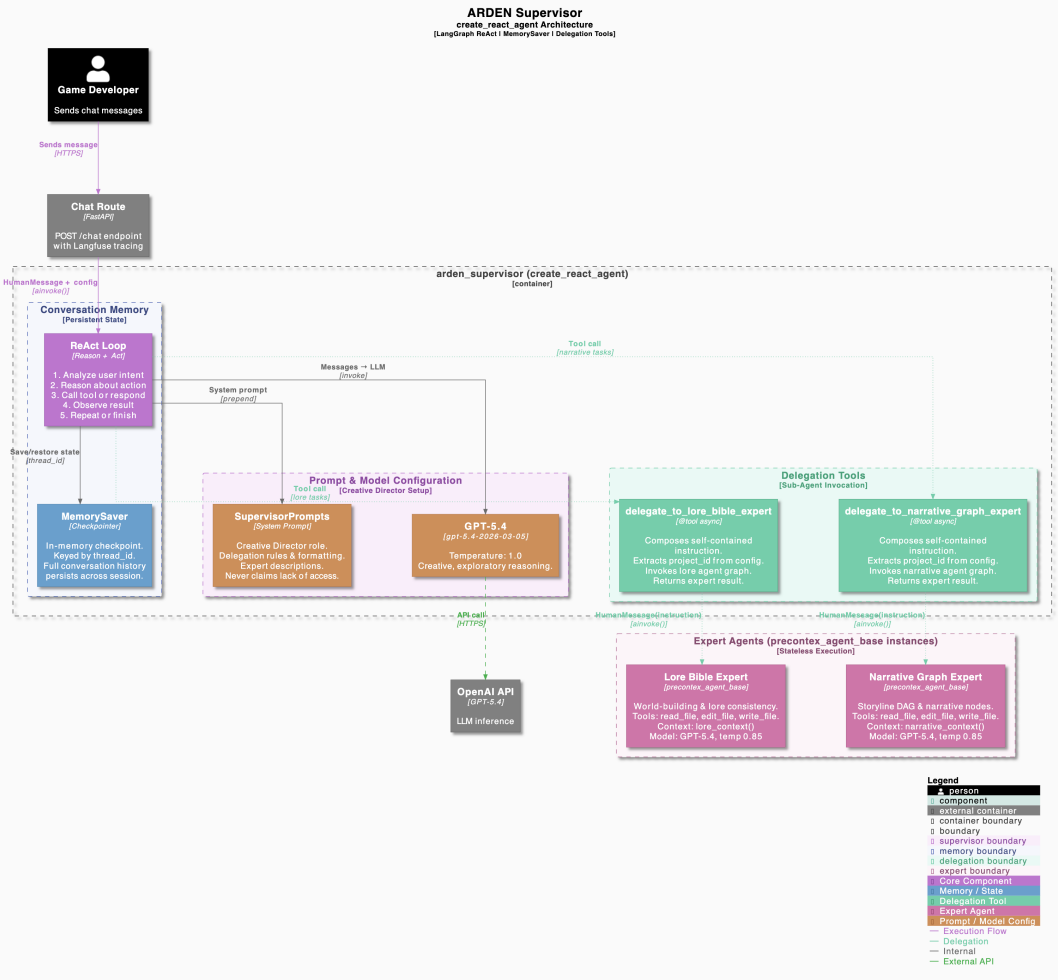

Figure 2: C4 Component Diagram: ARDEN Supervisor — built with LangGraph's `create_react_agent` (ReAct pattern) and a `MemorySaver` checkpointer for persistent conversation memory. Its tools are delegation functions (`delegate_to_lore_bible_expert`, `delegate_to_narrative_graph_expert`) that invoke `precontex_agent_base` instances with self-contained instructions composed from the conversation context. Model: GPT-5.4, temperature 1.0.

D. Agentic Workflow

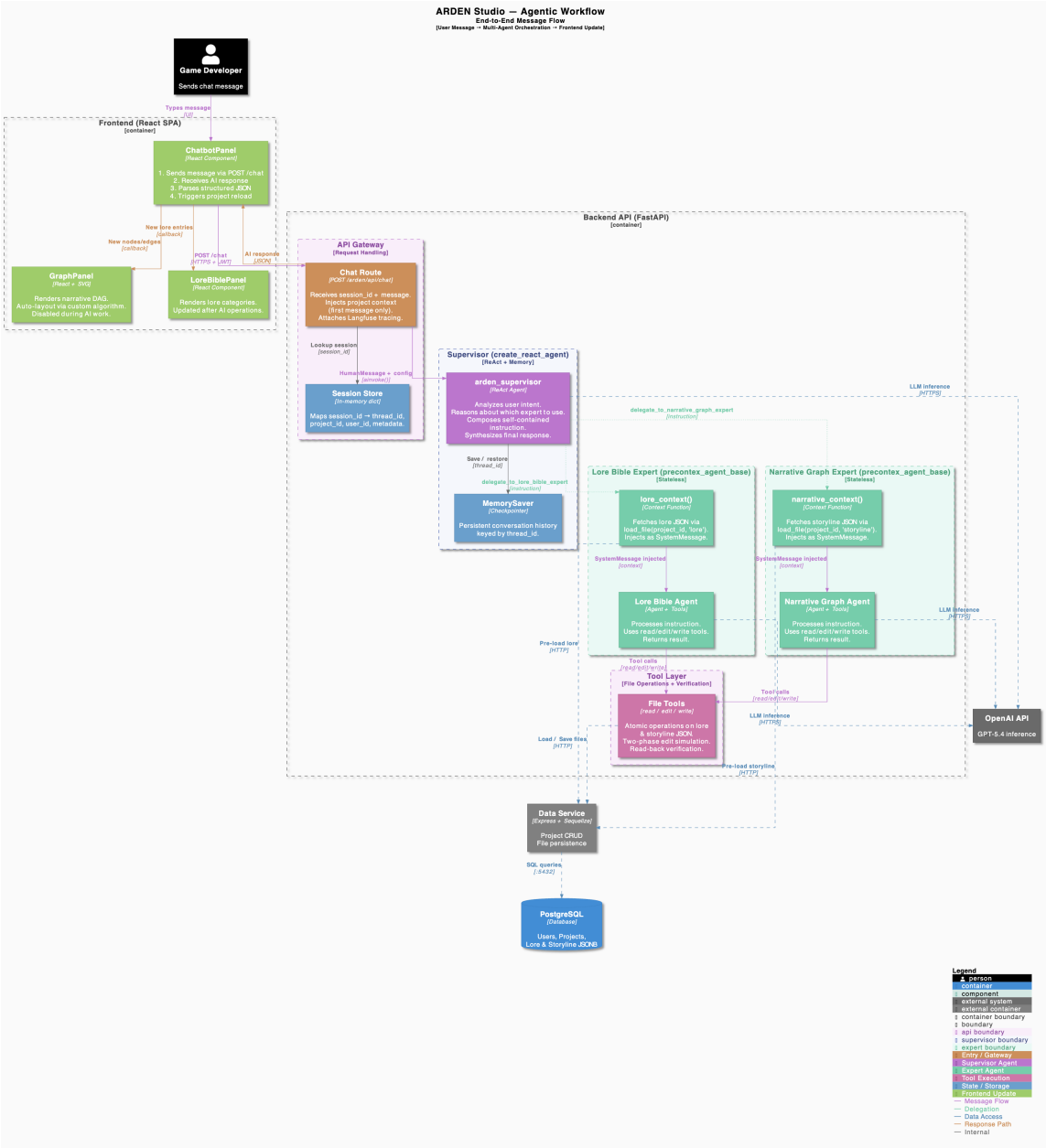

Figure 3: End-to-end agentic workflow: User message → Chat Route → Supervisor (ReAct + MemorySaver) → Delegation → Expert Sub-Agent (context injection + tool execution loop) → Result return → Supervisor synthesis → Frontend update. Three independent LLM instances with per-agent prompt engineering, model configuration, and tool bindings. Supervisor owns memory and intent; experts own tools and domain execution; context injection bridges stateless experts to project data.

E. Backend API Components

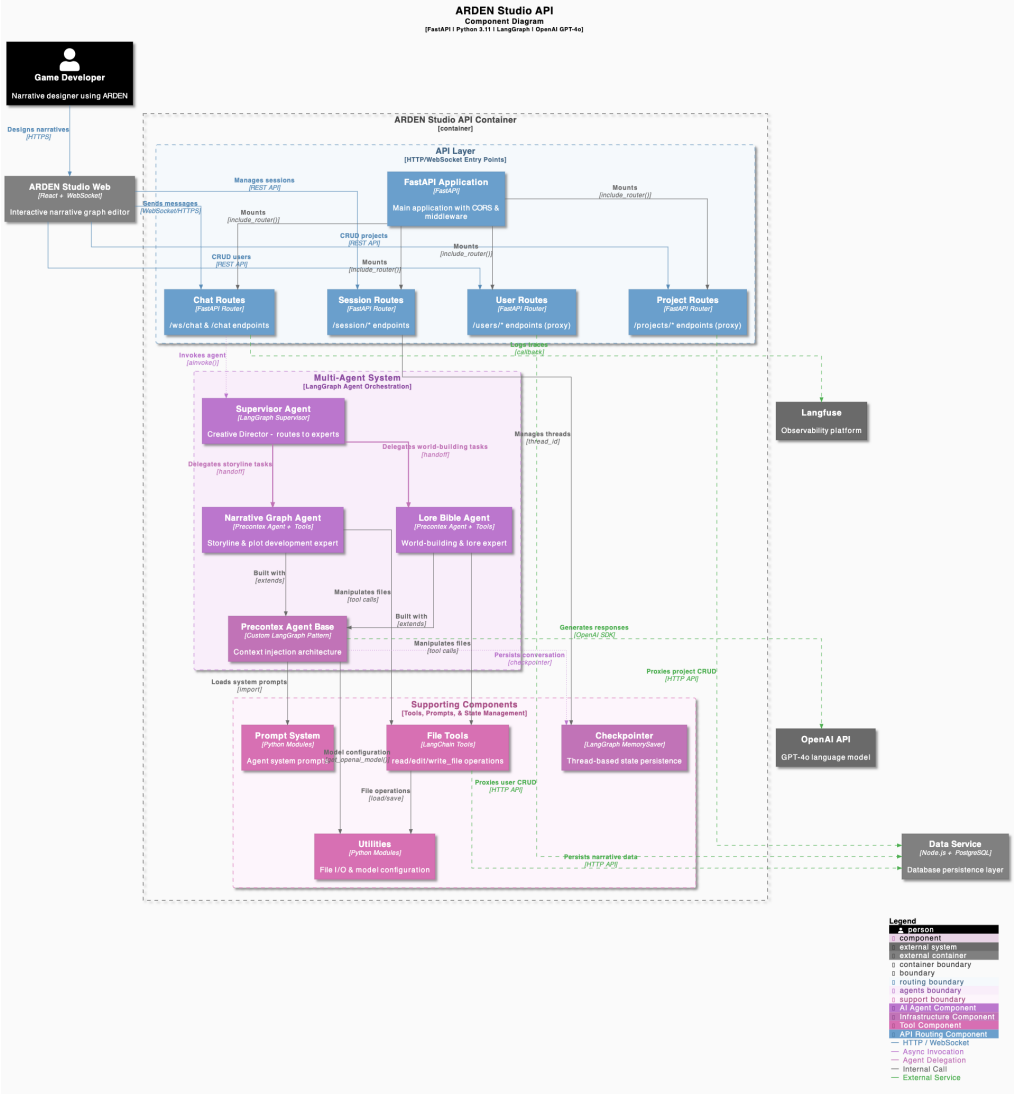

Figure 4: C4 Component Diagram: Backend API architecture showing the multi-agent system, session management, and data service integration.

F. Frontend Components

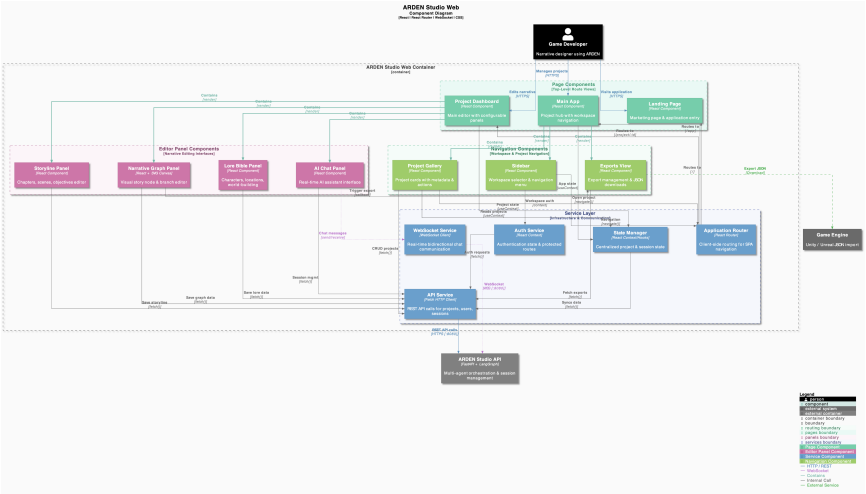

Figure 5: C4 Component Diagram: Frontend application with graph editor, AI assistant, lore bible, and navigation components.

G. Data Service Components

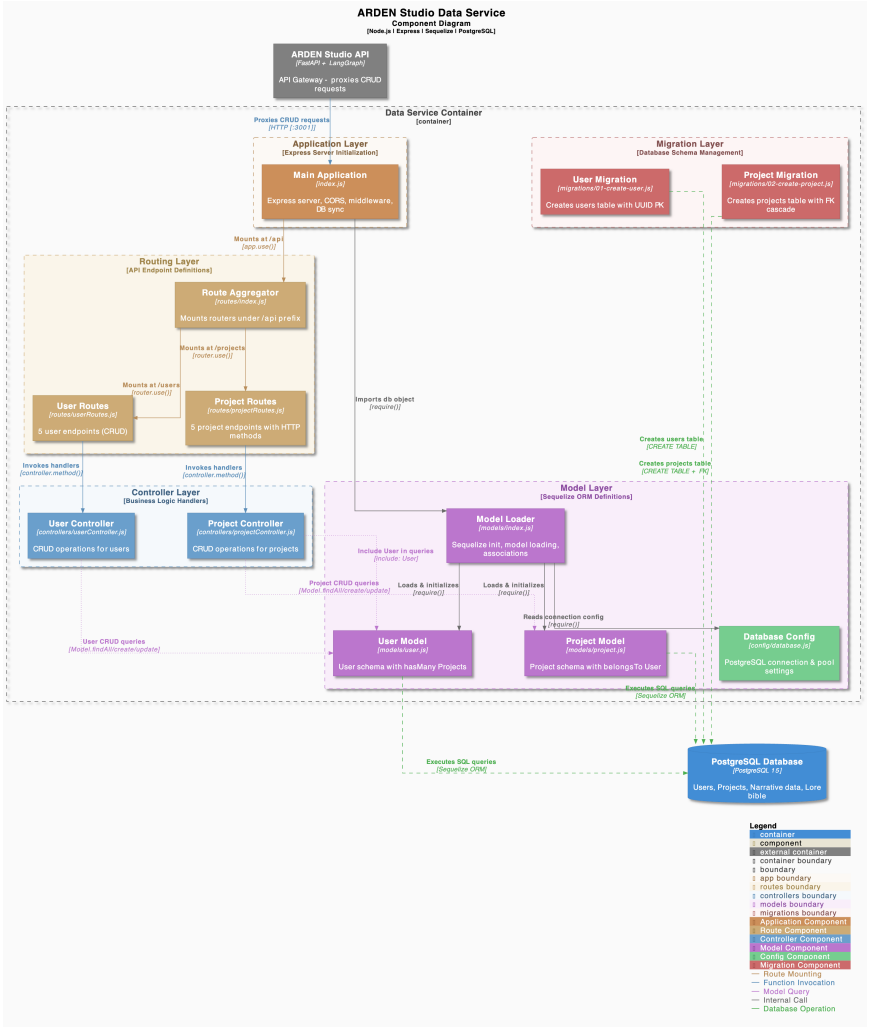

Figure 6: C4 Component Diagram: Data Service layer with routes, controllers, and Sequelize models.

H. Data Service Class Diagram

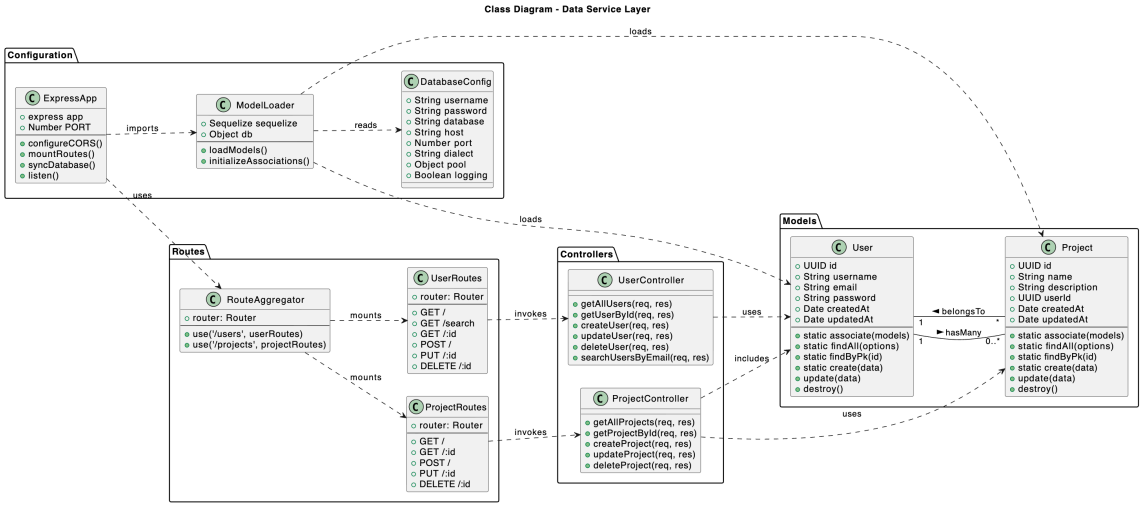

Figure 7: Class Diagram: Data Service showing routes, controllers, models, and Sequelize associations.